Programació 3

Guia pas a pas de la Part-1 Projecte 2026-27

Descripció de la Guia

- Abans de seguir aquesta guia cal **llegir completament el document** de l'enunciat del projecte per entendre l'abast del mateix i el que es demana.
- Per seguir la guia cal instal·lar l'entorn de desenvolupament, que ha de ser **Visual Studio 2022** (Community Edition o Professional), amb els paquets de **C++** i **C#** amb el Runtime **.NET 8** que s'especifiquen dins l'enunciat.
- En aquesta guia es descriuen aspectes tècnics del setup i de les tasques a realitzar i implementar.

Divisió en dos blocs

- Bloc 1: Per practicar i entendre la informació que teniu:
  - **Projecte codi Tasques 1-i-2 (VS-2022) (ZIP)**: En aquest projecte es realitzen les tasques 1 i 2; que serveixen només per conèixer com OSM estructura les dades. Fer això us facilitarà entendre les tasques posteriors i no s'avaluarà.
- Bloc 2: El projecte per lliurar de la Part 1:
  - **Projecte codi Gràfic Part 1 (VS-2022) (ZIP)**: Aquest projecte us servirà per comprovar que els punts i camins que esteu definint són els correctes de manera visual i fer debug. Teniu que començar a realitzar les tasques aquí.
  - **Avaluador Projecte – Part 1 – Grup XXX**: Entrega a Caronte a través del vostre grup del projecte. Aquest és el codi responsable de la qualificació, que es realitza de manera automàtica. La nota sortirà (exclusivament) d'aquest codi lliurat!

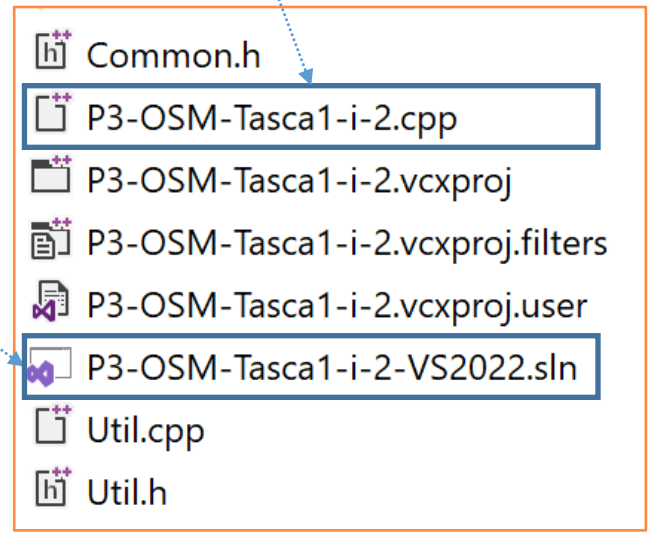
Tasques 1 i 2

Aquestes tasques serveixen per familiaritzar-vos amb l'entorn que vos donem.

Primer cal descarregar el projecte a Caronte: **VSCE2022 - ALUMNES P1 (Tasca1-i-2) - 2027.zip**

Allà està el projecte Visual Studio, a on dins el codi de "LP-OSM-Tasca1-i-2.cpp" hi ha dos objectes de tipus **XmlElement** (la classe XmlElement està definida dins "Common.h") tals que:

- Un representa un node d'interès: ***viejoRoble***
- Un altre representa un camí: ***camiJoaquim***



Tasca 1

Iterar **viejoRoble** per tenir
id, *lat* i *lon*, per Tasca 1

```
XmlElement viejoRoble = {
     //id_element =
     "node",
     // atributs =
     {
          std::make_pair("id", "357101444"),
          std::make_pair("visible", "true"),
          // ... Codi restant. Veure complert a estudiants-tasca1-2
          std::make_pair("uid", "4946691"),
          std::make_pair("lat", "41.4902204"),
          std::make_pair("lon", "2.1406477")
     },
     // fills =
     {
          std::make_pair<std::string,std::vector<std::pair<std::string, std::string>>>
          ("tag", {
          std::make_pair<std::string, std::string>("k", "amenity"),
          std::make_pair<std::string, std::string>("v", "restaurant")
          }),
     // ... Codi restant. Veure complert a estudiants-tasca1-2
     }
};
```

Tasca 1:
Imprimir per pantalla ident, latitud (lat), longitud (lon) i nom (name) de l'element XML viejoRoble

Transformar latitud i longitud a double.

A la pràctica haureu de recollir tots els punts d'interès (nodes que no són nodes de camí que tenen un atribut name).

Tasca 2

Per **camiJoaquim**, a Tasca 2, imprimir els identificadors dels nodes que el formen

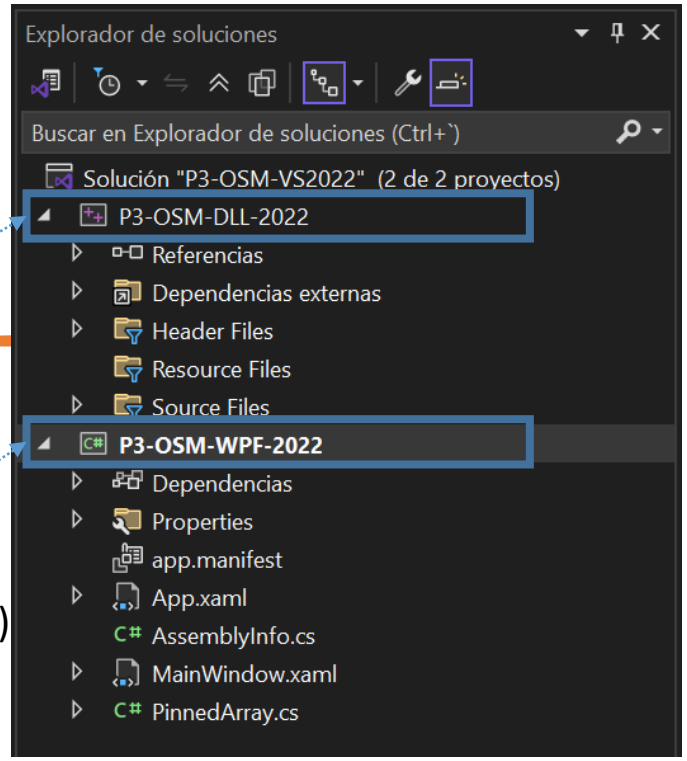
```
XmlElement camiJoaquim = {
// id_element ="way",
// atributs =
{std::make_pair("id", "31889094"),
 std::make_pair("visible", "true"),
 std::make_pair("version", "6"),
//…
},
// fills =
{std::make_pair<std::string, std::vector<std::pair<std::string, std::string>>>
("nd",{std::make_pair("ref", "357125056")}),
//…
std::make_pair<std::string, std::vector<std::pair<std::string, std::string>>>
("tag", {std::make_pair("k", "highway"),std::make_pair("v", "living_street")}),
std::make_pair<std::string, std::vector<std::pair<std::string, std::string>>>
("tag", {std::make_pair("k", "name"),std::make_pair("v", "Carrer de Sant Joaquim")}),
//…
};
```

Tasca 2:
Imprimir identificadors dels nodes que formen XmlElement camiJoaquim, i valor atribut highway.

A la pràctica només tots els camins amb tag highway i saber quins nodes de tipus camí, formen cada camí.

```
valorTag = Util::kvDeTag() //valorTag.first ‘highway’; valorTag.second ‘living_street’
```

Tasques 3-6 Projecte



Projecte codi Gràfic Part 1 (VS-2022) (ZIP): La solució té dos sub-projectes:

- **P3-OSM-DLL-2022**: Inclou la part de càlcul i estructures de dades
  - Està escrit en C++, i és el projecte a <u>on teniu que implementar la vostra solució</u>
  - D'aquí sortirà el codi que vosaltres lliurareu a Caronte (copiant els arxius font .cpp/.h)
- **P3-OSM-WPF-2022**: Presenta l'entorn visual i està escrit en C# WPF
  - Incorpora l'altre sub-projecte i el compila, a més de ser el projecte d'execució principal
  - **NO heu de modificar res de cap fitxer d'aquest projecte per assolir els objectius de la pràctica**

- Només teniu que fer build/rebuild del projecte WPF i podreu executar/debuggar el vostre codi
- El vostre codi es compila dins "[carpeta_projecte]\bin\P3-OSM-DLL-2022\"
- Però l'executable es generarà a dins de (segons Architecture [x64|x86|arm64] y Target [Debug|Release]) **"[carpeta_projecte]\bin\P3-OSM-WPF-2022\bin\x64\Debug\net8.0-windows\P3-OSM-WPF-2022.exe"**
- Per fer un 'clean' total només cal esborrar el directori "[carpeta_projecte]\bin\"

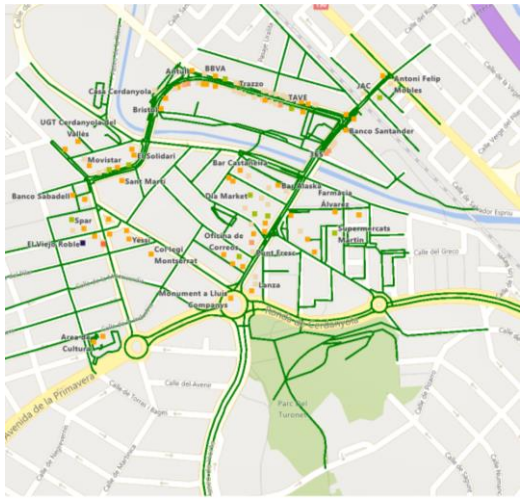
Flux de coordinació dels dos paquets

P3-OSM-DLL-2022

Genera una DLL, que crida als vostres mètodes

Importa la DLL, transferint les dades que retorneu

P3-OSM-WPF-2022



Representació gràfica

IMPORTANT: Per això és que cal que utilitzeu **herència** i que **implementeu els mètodes tal com els us demanem**!

Si no es fa així, llavors els dos projectes no s'entendran correctament i no funcionaran, fent impossible el debug

Tasca 3: PuntDeInteresBase

Tenim diferents punts d'interès, però la classe base ens indica una interface comuna a tots:

```
class PuntDeInteresBase {
    private:
        Coordinate m_coord;
        std::string m_name;

    public:
        PuntDeInteresBase();
        PuntDeInteresBase(Coordinate coord, std::string name);

        virtual std::string getName();
        Coordinate getCoord();
        virtual unsigned int getColor();
};
```

Crear classes derivades **PuntDeInteresBotigaSolucio** i **PuntDeInteresRestaurantSolucio**

Implementar mètodes a les classes derivades, segons als atributs descrits a l'enunciat

Coordinate: Estructura amb dos camps: lat i lon, que defineix la posició en el mapa. Definida a Common.h

```
typedef struct {
    double lat;
    double lon;
} Coordinate;
```

Tasca 3: PuntDeInteresBotigaSolucio

```
class PuntDeInteresBase {
    private:
        Coordinate m_coord;
        std::string m_name;

    public:
        PuntDeInteresBase();
        PuntDeInteresBase(Coordinate coord,
                          std::string name);

        virtual std::string getName();
        Coordinate getCoord();
        virtual unsigned int getColor();
};
```

Per aquesta subclasse s'han d'afegir 3 atributs:

- El valor del tag "shop"
- El valor del tag "opening_hours"
- El valor de "wheelchair"

Retornarà:

1. 0xDFFF00 pels "supermarket"
2. 0xFF7F50 pels "tobacco"
3. 0xFA8072 pels "bakery".
4. 0x4CBB17 pels "bakery" amb "opening_hours" que contingui el text "06:00-22:00" amb "wheelchair" igual a "yes" (accés per persones amb mobilitat reduïda).
5. 0xFFEA00 en els altres casos (altres "shop")

Per la definició d'altres colors, podeu utilitzar https://coolors.co/generate

Tasca 3: PuntDeInteresRestaurantSolucio

```
class PuntDeInteresBase {
    private:
        Coordinate m_coord;
        std::string m_name;

    public:
        PuntDeInteresBase();
        PuntDeInteresBase(Coordinate coord, std::string name);

        virtual std::string getName();
        Coordinate getCoord();
        virtual unsigned int getColor();
};
```

En aquesta subclasse són dos atributs addicionals:

- Tipus de cuina ("cuisine")
- Si facilita accés a persones amb mobilitat reduïda

Retornarà:

1. **0x7FFFD4** pels "pizza"+ mobilitat reduïda
2. **0x00FFFF** pels "chinese"
3. **0x5D3FD3** per la resta amb mobilitat reduïda ("wheelchair").
4. **Color definit per defecte** en els altres casos

Nota: Tingueu en compte que el color per defecte ve definit per PuntDeInteresBase, i pot es canviar o fer hardcode.

Tasca 4: CamiBase

Volem tenir definits els camins:

```
#include "Common.h"
#include <vector>

class CamiBase {
    public:
        virtual std::vector<Coordinate> getCamiCoords() = 0;
};
```

Crear classe
Derivada
CamiSolucio

Implementar mètode, i definir atributs i
mètodes extra a la classe derivada
Segons sigui necessari
per afegir les coordenades per
MapaSolucio

Tasca 4: CamiSolucio

Volem tenir definits els camins:

```
#include "Common.h"
#include <vector>

class CamiBase {
    public:
        virtual std::vector<Coordinate> getCamiCoords() = 0;
};
```

Crear clase Derivada **CamiSolucio**
Implementa mètode getCamiCoords()
Coordinate: Estructura amb dos camps: lat i lon, que defineix la posició en el mapa d'un node del camí.

Tasca 4: Prova aïllada per visualitzar un primer camí que fa que getCamiCoords() retorni el següent conjunt de coordenades pre-definit de (lat, lon)

- (41.4928803, 2.1452381)
- (41.4929072, 2.1452474)
- (41.4933070, 2.1453852)
- (41.4939882, 2.1456419)

Tasca 5: MapaSolucio

```
class MapaBase {

    public:
        virtual void getPdis(std::vector<PuntDeInteresBase*>&) = 0;

        virtual void getCamins(std::vector<CamiBase*>&) = 0;

        virtual void parsejaXmlElements(std::vector<XmlElement>&
                 xmlElements) = 0;

};
```

Crear classe derivada **MapaSolucio**

Agafar les dades de xmlElements per obtenir Punts de Interes i Camins, a MapaSolucio

Retornar tot el conjunt de camins parsejats, a MapaSolucio

Retornar tot el conjunt de Punts d'Interes parsejats, a MapaSolucio

Tasca 5: MapaSolucio

```
class MapaBase {
    public:
        virtual void getPdis(std::vector<PuntDeInteresBase*>&) = 0;
        virtual void getCamins(std::vector<CamiBase*>&) = 0;
        virtual void parsejaXmlElements(std::vector<XmlElement>& xmlElements) = 0;
};
```

Crear classe derivada **MapaSolucio**

Vosaltres haureu de crear una classe derivada de **MapaBase**, que es dirà **MapaSolucio** i que guardarà la seva informació com a estructures internes. Aquestes estructures les haureu de pensar, dissenyar i implementar perquè siguin òptimes, i guardaran la informació del fitxer XML (punts d'interès i camins formats per nodes).

IMPORTANT: En utilitzar la interfície gràfica, els mètodes són cridats mitjançant les funcionalitats del GUI. Es recomana SEMPRE utilitzar Fitxer >> Obrir, carregar un fitxer, i llavors, utilitzar els botons per visualitzar la informació corresponent i cridar així l'execució dels vostres mètodes.

Tasca 5: MapaSolucio

```
class MapaBase {
    public:
        virtual void getPdis(std::vector<PuntDeInteresBase*>&) = 0;
        virtual void getCamins(std::vector<CamiBase*>&) = 0;
        virtual void parsejaXmlElements(std::vector<XmlElement>& xmlElements) = 0;
};
```

Tasca 5: Implementeu el mètode getPdis(std::vector<PuntDeInteresBase*>&)

1. Retorna dos punts d'interès (que heu implementat a la **Tasca 3**):

- un **PuntDeInteresBotigaSolucio:** ***shop*** valor "***bakery***" nom "La Millor Pastisseria" situat a (41.4918606, 2.1465411)
- un **PuntDeInteresRestaurantSolucio**, ***amenity*** valor "***restaurant",*** cuina "***regional***" i mobilitat reduïda. Situat a (41.4902204, 2.1406477) i nom "El Millor Restaurant".

```
MapaRender::MapaRender()
{   this->m_xml4osm_util = new XML4OSMUtil();
    // TODO: instancieu m_mapaBase amb la vostra MapaSolucio
    this->m_mapaBase = nullptr;
}
```

Nota: Instancieu correctament la propietat m_mapaBase de la classe **MapaRender** construint la vostra solució de **MapaSolucio**

Tasca 6: MapaSolucio

```
class MapaBase {
    public:
        virtual void getPdis(std::vector<PuntDeInteresBase*>&) = 0;
        virtual void getCamins(std::vector<CamiBase*>&) = 0;
        virtual void parsejaXmlElements(std::vector<XmlElement>& xmlElements) = 0;
};
```

Tasca 6: Retorneu el camí que heu implementat a la **Tasca 4** mitjançant la implementació de getCamins(std::vector<CamiBase*>&)

- **void getCamins(std::vector<CamiBase*>& camins)**
  - Retorna tots els camins que haureu parsejat i que són **highway**.
  - Nota: La **classe CamiBase** és una classe virtual pura i necessitareu definir una classe derivada **CamiSolucio** on afegireu nous atributs i funcionalitats.

Tasques1-6 amb Interfície Gràfica

Un cop realitzades correctament les tasques abans descrites, ja podreu començar a visualitzar punts d'interès i camins utilitzant la interfície gràfica. Però encara no haureu llegit cap fitxer OSM. Així que si heu implementat correctament els mètodes i atributs detallats, llavors el vostre resultat hauria de representar dos punts d'interès i un camí a la regió de Cerdanyola.

Al Constructor de MapaRender, cal que instancieu m_mapaBase **amb el vostre MapaSolucio**

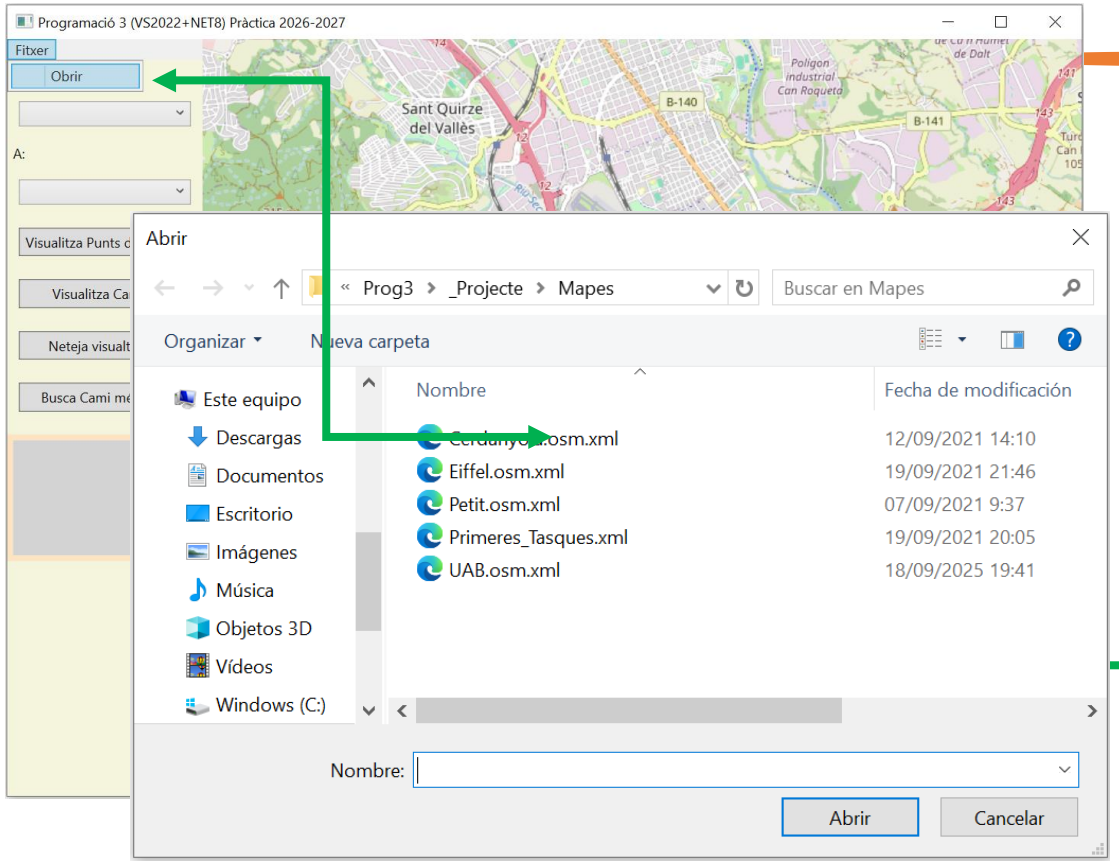
```
// … MapaRender.h codi
MapaRender::MapaRender() {

    this->m_xml4osm_util = new XML4OSMUtil();

    // TODO: Cal que instancieu m_mapaBase amb la vostra MapaSolucio
    this->m_mapaBase = nullptr;
}

// … MapaRender.h codi
```

Agafant dades reals de OSM...




```
void parsejaXmlElements(std::vector<XmlElement>& xmlElements)
```

Llegeix les dades del vector construint les estructures internes **(Tasca 7)** per guardar: punts d'interés i camins

`Informació real que prové del XML en format de vector de XmlElement. Veure Tasca 1 i Tasca 2.`

```
void getPdis(std::vector<PuntDeInteresBase*>&)
```

Retorna a la interficie gràfica els punts d'interès desats a l'estructura interna

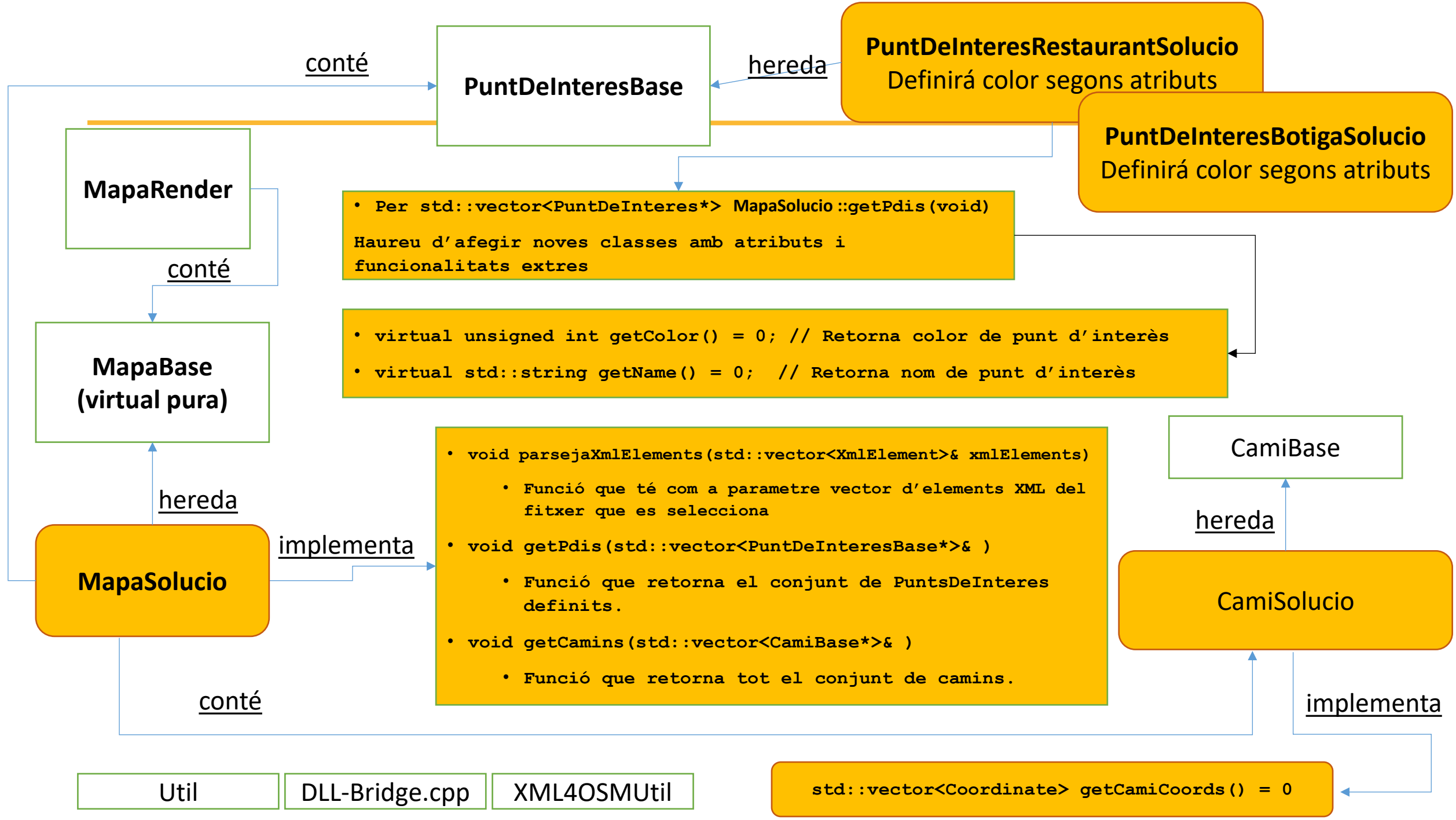
Tasca 8

```
void getCamins(std::vector<CamiBase*>&)
```

Retorna a la interficie gràfica els camins desats a l'estructura interna

Tasca 9

Modificar la classe MapaSolucio per emmagatzemar les dades a partir d'estructures dinàmiques internes: Definició de l'estructura de dades interna necessària **Tasca 7**


conté
PuntDeInteresBase
hereda
PuntDeInteresRestaurantSolucio
Definirá color segons atributs
PuntDeInteresBotigaSolucio
Definirá color segons atributs
MapaRender
conté
• Per std::vector<PuntDeInteres*> MapaSolucio::getPdis(void)
Haureu d'afegir noves classes amb atributs i funcionalitats extres
MapaBase (virtual pura)
• virtual unsigned int getColor() = 0; // Retorna color de punt d'interès
• virtual std::string getName() = 0; // Retorna nom de punt d'interès
hereda
implementa
MapaSolucio
• void parsejaXmlElements(std::vector<XmlElement>& xmlElements)
• Funció que té com a parametre vector d'elements XML del fitxer que es selecciona
• void getPdis(std::vector<PuntDeInteresBase*>&)
• Funció que retorna el conjunt de PuntsDeInteres definits.
• void getCamins(std::vector<CamiBase*>&)
• Funció que retorna tot el conjunt de camins.
CamiBase
hereda
CamiSolucio
conté
implementa
Util
DLL-Bridge.cpp
XML4OSMUtil
std::vector<Coordinate> getCamiCoords() = 0

Common.h

<< Common.h >>

```
typedef std::pair<std::string, std::string> PAIR_ATTR_VALUE;
typedef std::pair<std::string, std::vector<PAIR_ATTR_VALUE>> CHILD_NODE;

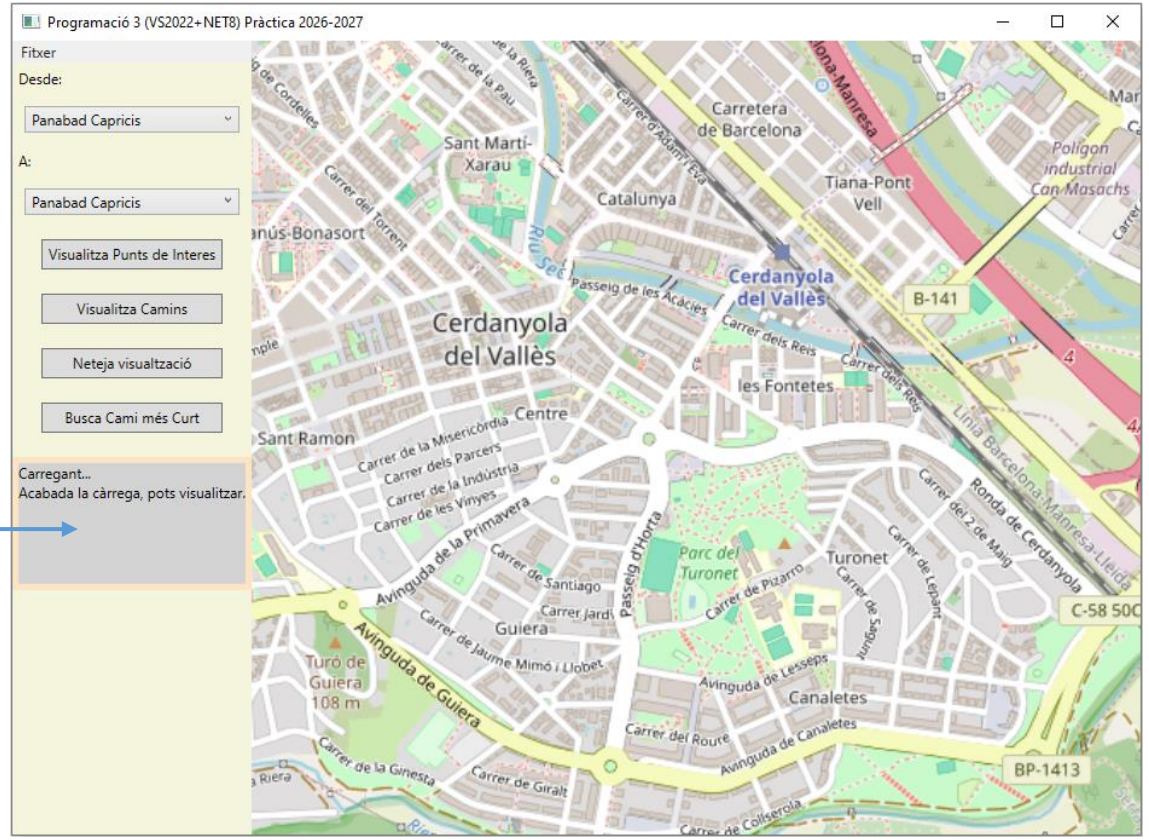
typedef struct {
    double lat;
    double lon;
} Coordinate;

typedef struct {
        std::string id_element;
        std::vector<PAIR_ATTR_VALUE> atributs;
        std::vector<CHILD_NODE> fills;
} XmlElement;
```

Classe Util

1. `Util::escriuEnMonitor(std::string text)`: permet escriure en un arxiu "Log.txt" el text que escriviu pel paràmetre ***text***.

NOTA:
El Monitor només imprimeix missatges del WPF



Classe Util

2. `Util::kvDeTag(std::vector<PAIR_ATTR_VALUE>& atributsTag)`: us permet llegir els sub-elements de tipus **<tag>** de manera fàcil.

Aquesta funció s'encarrega d'iterar tots els atributs del tag (és a dir, k="clau" v="valor"), i en retorna un parell on el primer valor es "clau" i el segon "valor".

Com a paràmetre, heu de facilitar-li tots els atributs del tag que esteu computant. Torneu a fer una ullada a l'exemple donat just abans de la descripció de la Tasca 1.

Entrega a Caronte

Una vegada que hagueu implementat les vostres funcionalitats dins el projecte Visual amb la interfície gràfica, haureu de fer el lliurament de la vostra implementació a la plataforma de Caronte.

Recordeu: L'ús de la interfície gràfica no s'utilitza per l'avaluació de la nota al projecte de l'assignatura. És més, podeu treballar directament amb els fitxers que se us proporciona a Caronte per l'avaluació automàtica. Tant mateix, sense visualitzador, segurament vos serà difícil comprovar que camins i punts d'interès són inicialment els correctes, i Caronte conté els seus propis tests.

Recordeu: Tot i que en aquesta guia pas a pas se us faciliten les tasques que heu de fer, la descripció detallada la trobareu a l'enunciat del projecte. Per qualsevol dubte, recordeu que a més de l'ajuda del professorat, teniu un fòrum especial pel projecte a Caronte per el seu bon ús.